11 / SLIDE 001 Deployment, container e CI/CD

Lezione: Deployment, container e CI/CD

Fonte: 11_deployment.pdf, pagina 1 di 13

Spiegazione per studiare

L'ultima lezione riguarda il passaggio dall'applicazione in sviluppo a un servizio utilizzabile dagli utenti. Presenta ambienti separati, metodi di distribuzione, container, Docker Compose, CI/CD e misure di sicurezza per la produzione.

Definizioni, passaggi ed esempi presenti nella slide

Deployment

SLIDE ORIGINALE · diagrammi, tabelle e codice







11 / SLIDE 002 Cos’è il Deployment

Lezione: Deployment, container e CI/CD

Fonte: 11_deployment.pdf, pagina 2 di 13

Spiegazione per studiare

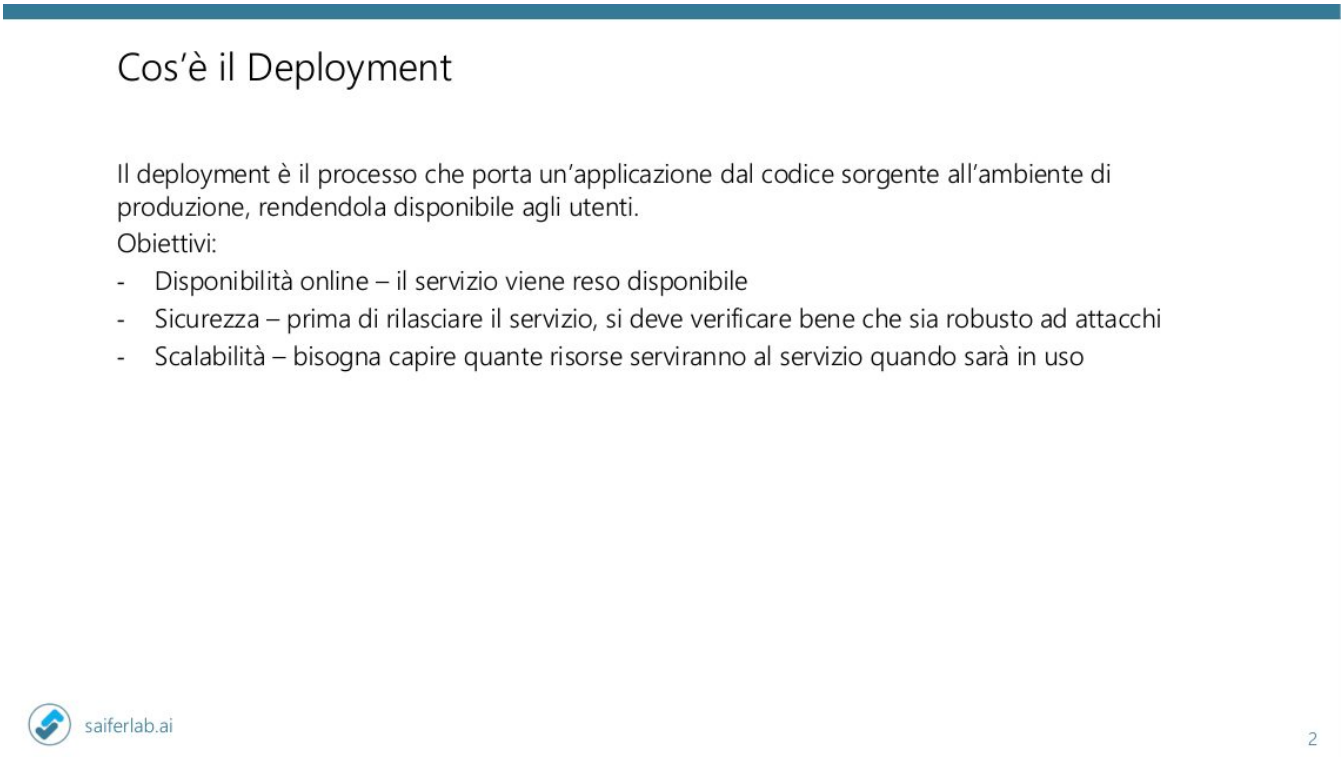
Il deployment comprende le attività che rendono il software disponibile in produzione. Non coincide con la semplice copia del codice: è necessario considerare disponibilità online, sicurezza del servizio e risorse necessarie quando arrivano richieste reali.

Definizioni, passaggi ed esempi presenti nella slide

Il deployment è il processo che porta un’applicazione dal codice sorgente all’ambiente di produzione, rendendola disponibile agli utenti. Obiettivi:

- Disponibilità online – il servizio viene reso disponibile
- Sicurezza – prima di rilasciare il servizio, si deve verificare bene che sia robusto ad attacchi
- Scalabilità – bisogna capire quante risorse serviranno al servizio quando sarà in uso

SLIDE ORIGINALE · diagrammi, tabelle e codice



11 / SLIDE 003 Ambienti di Esecuzione

Lezione: Deployment, container e CI/CD

Fonte: 11_deployment.pdf, pagina 3 di 13

Spiegazione per studiare

Sviluppo, test/staging e produzione sono ambienti con obiettivi differenti. I cambiamenti si preparano e verificano in un ambiente isolato prima di essere distribuiti agli utenti, così da evitare che esperimenti o malfunzionamenti interrompano il servizio attivo.

Definizioni, passaggi ed esempi presenti nella slide

È bene avere ambienti separati per evitare errori catastrofici

• per esempio, lo sviluppo di nuove funzionalità deve essere ben isolato dal servizio che è attivo e usato dagli utenti al momento

• Gli utenti saranno avvisati quando si aggiornerà il codice sul servizio attivo, ma questo viene fatto solo dopo aver verificato che funzioni tutto in un ambiente di pre-deployment

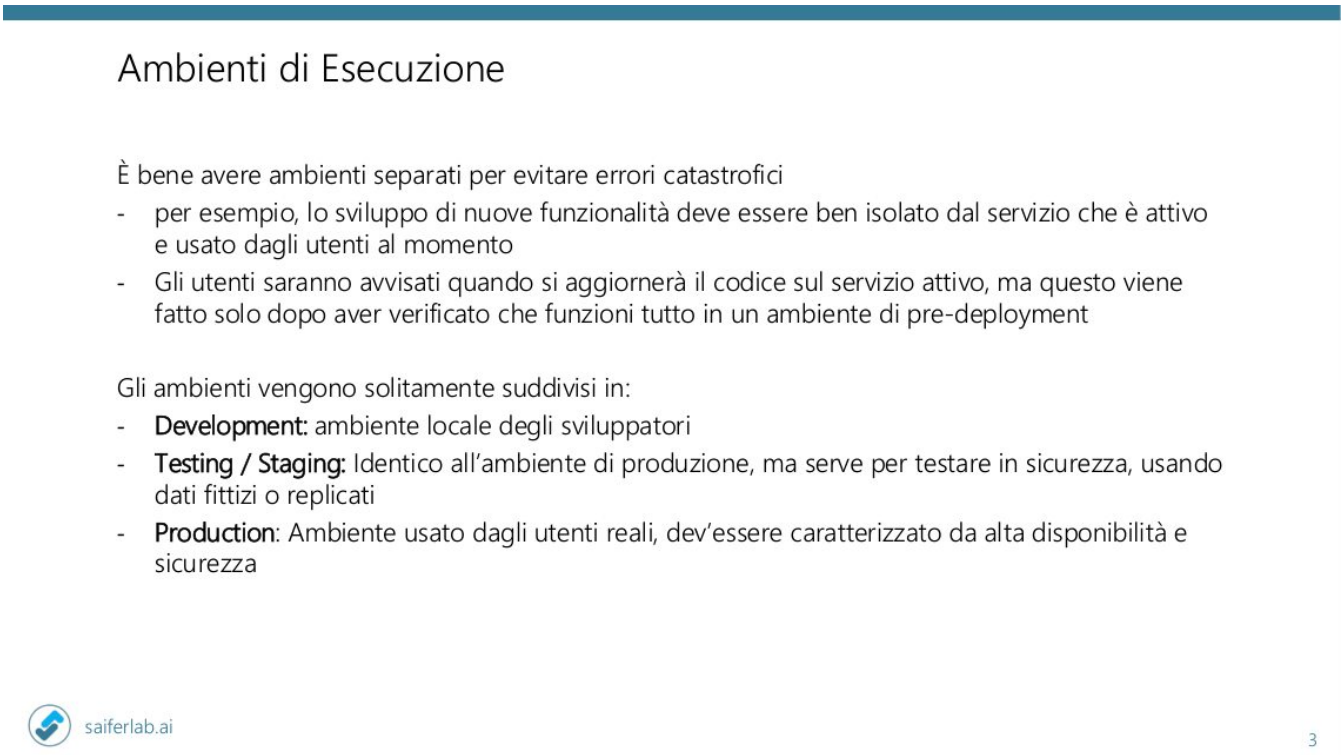
Gli ambienti vengono solitamente suddivisi in:

• Development: ambiente locale degli sviluppatori

• Testing / Staging: Identico all'ambiente di produzione, ma serve per testare in sicurezza, usando dati fittizi o replicati

• Production: Ambiente usato dagli utenti reali, dev'essere caratterizzato da alta disponibilità e sicurezza

SLIDE ORIGINALE · diagrammi, tabelle e codice



11 / SLIDE 004 Metodi di deployment

Lezione: Deployment, container e CI/CD

Fonte: 11_deployment.pdf, pagina 4 di 13

Spiegazione per studiare

Il deployment manuale usa strumenti come FTP, SFTP e SSH per trasferire file e lanciare comandi sul server. È semplice da iniziare ma espone a errori e scarsa tracciabilità; gli script rendono invece più ripetibili le operazioni, a costo di mantenerli.

Definizioni, passaggi ed esempi presenti nella slide

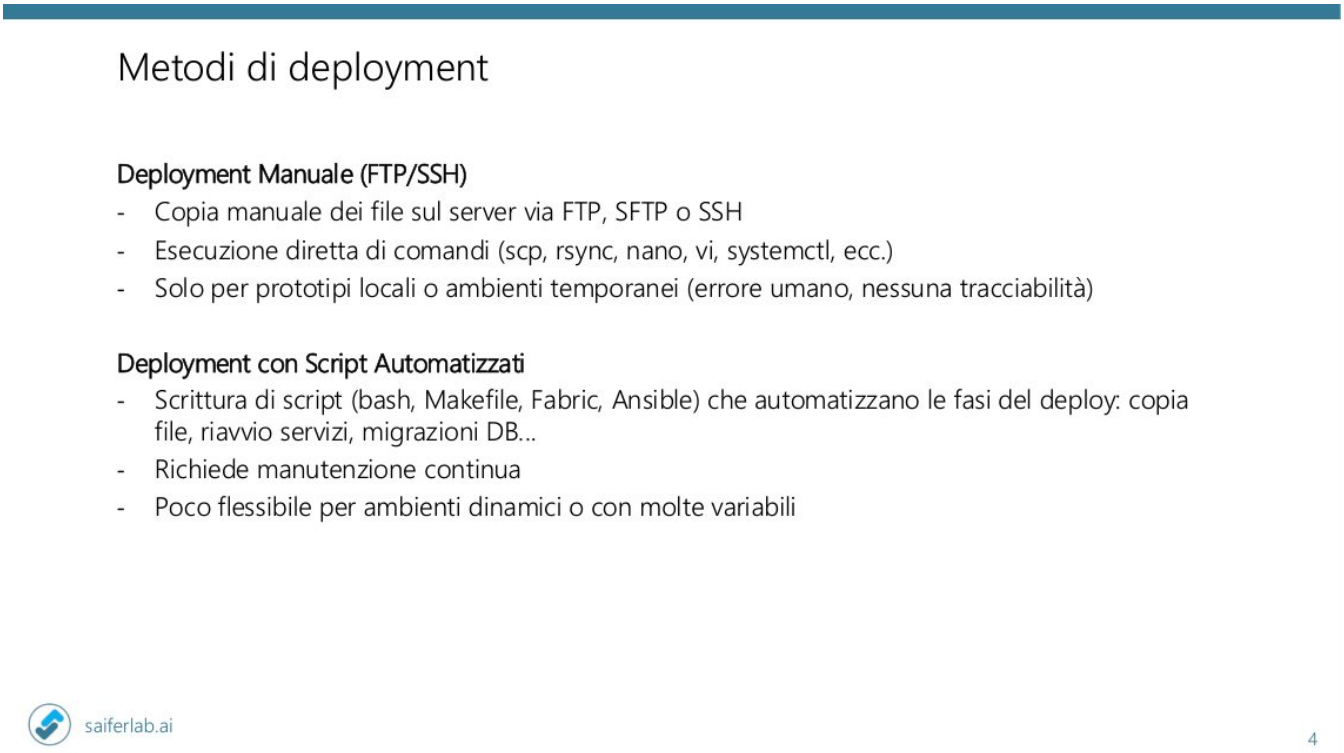
Deployment Manuale (FTP/SSH)

- Copia manuale dei file sul server via FTP, SFTP o SSH
- Esecuzione diretta di comandi (scp, rsync, nano, vi, systemctl, ecc.)
- Solo per prototipi locali o ambienti temporanei (errore umano, nessuna tracciabilità)

Deployment con Script Automatizzati

- Scrittura di script (bash, Makefile, Fabric, Ansible) che automatizzano le fasi del deploy: copia file, riavvio servizi, migrazioni DB...
- Richiede manutenzione continua
- Poco flessibile per ambienti dinamici o con molte variabili

SLIDE ORIGINALE · diagrammi, tabelle e codice



11 / SLIDE 005 Metodi di deployment

Lezione: Deployment, container e CI/CD

Fonte: 11_deployment.pdf, pagina 5 di 13

Spiegazione per studiare

Le pipeline CI/CD collegano il repository alle verifiche e al rilascio tramite strumenti come GitHub Actions. Docker crea ambienti applicativi confezionati con le dipendenze; Docker Compose ne coordina più di uno, ma la containerizzazione non sostituisce da sola l'automazione del rilascio.

Definizioni, passaggi ed esempi presenti nella slide

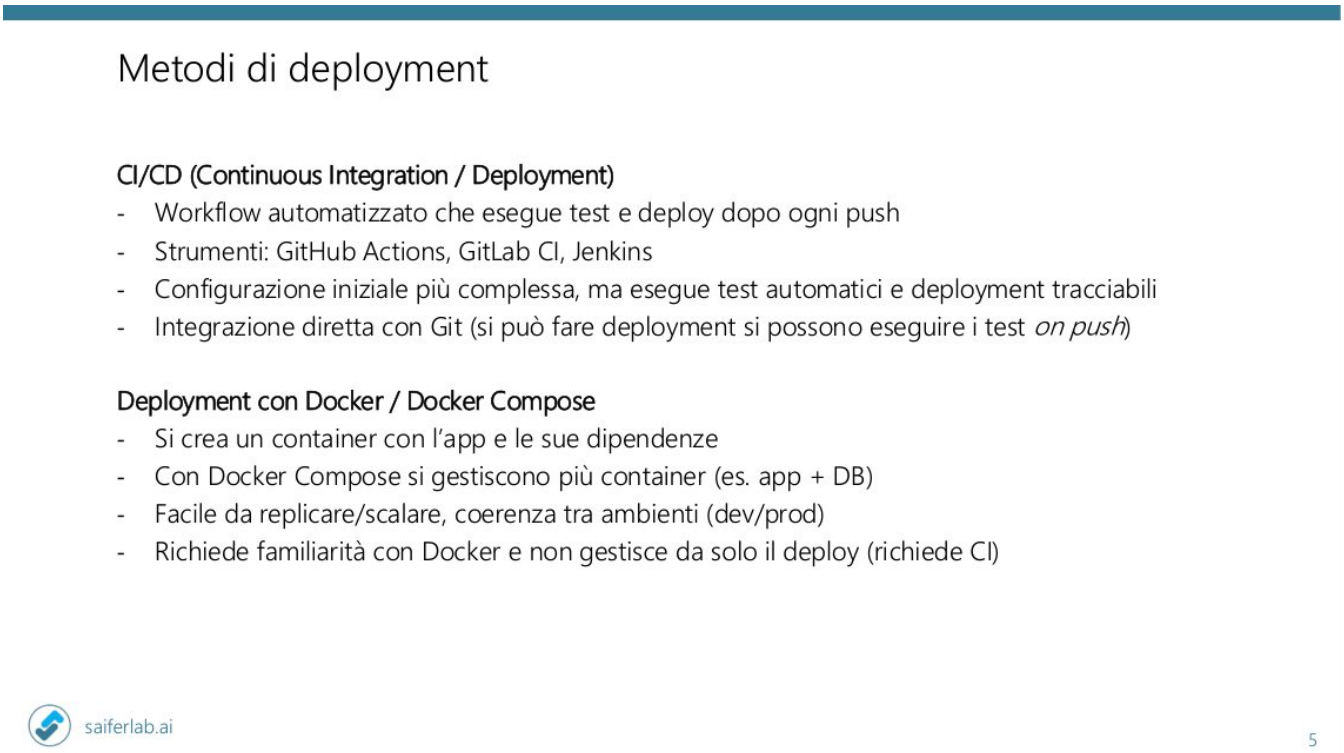
CI/CD (Continuous Integration / Deployment)

- Workflow automatizzato che esegue test e deploy dopo ogni push
- Strumenti: GitHub Actions, GitLab CI, Jenkins
- Configurazione iniziale più complessa, ma esegue test automatici e deployment tracciabili
- Integrazione diretta con Git (si può fare deployment si possono eseguire i test on push)

Deployment con Docker / Docker Compose

- Si crea un container con l'app e le sue dipendenze
- Con Docker Compose si gestiscono più container (es. app + DB)
- Facile da replicare/scalare, coerenza tra ambienti (dev/prod)
- Richiede familiarità con Docker e non gestisce da solo il deploy (richiede CI)

SLIDE ORIGINALE · diagrammi, tabelle e codice



11 / SLIDE 006 Metodi di deployment

Lezione: Deployment, container e CI/CD

Fonte: 11_deployment.pdf, pagina 6 di 13

Spiegazione per studiare

Con un PaaS si affida alla piattaforma parte della gestione di build e hosting, rinunciando a una quota di controllo. Con un IaaS si amministra un server virtuale, con maggior libertà ma anche responsabilità su rete, sicurezza, aggiornamenti e manutenzione.

Definizioni, passaggi ed esempi presenti nella slide

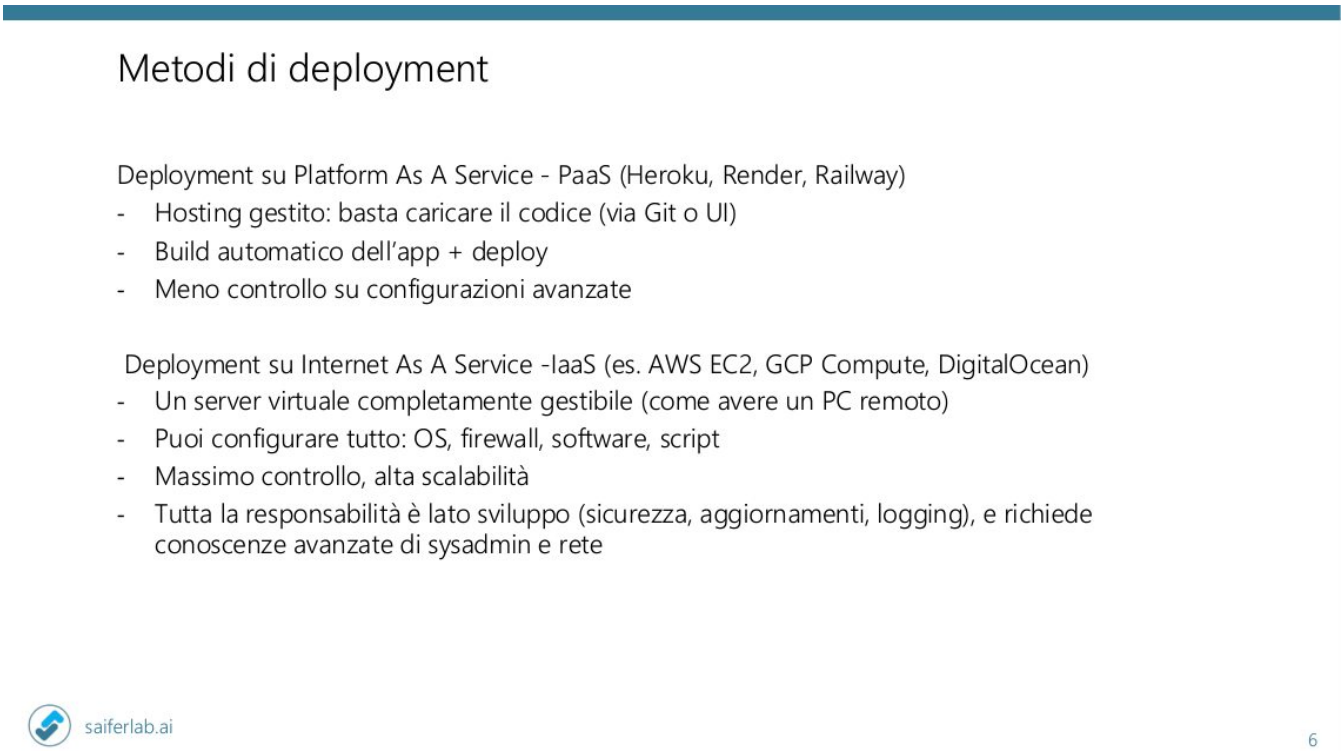
Deployment su Platform As A Service - PaaS (Heroku, Render, Railway)

- Hosting gestito: basta caricare il codice (via Git o UI)
- Build automatico dell'app + deploy
- Meno controllo su configurazioni avanzate

Deployment su Internet As A Service -IaaS (es. AWS EC2, GCP Compute, DigitalOcean)

- Un server virtuale completamente gestibile (come avere un PC remoto)
- Puoi configurare tutto: OS, firewall, software, script
- Massimo controllo, alta scalabilità
- Tutta la responsabilità è lato sviluppo (sicurezza, aggiornamenti, logging), e richiede conoscenze avanzate di sysadmin e rete

SLIDE ORIGINALE · diagrammi, tabelle e codice



11 / SLIDE 007 Metodi di deployment

Lezione: Deployment, container e CI/CD

Fonte: 11_deployment.pdf, pagina 7 di 13

Spiegazione per studiare

La tabella riassume vantaggi e svantaggi dei principali metodi di deployment. Il confronto mette in evidenza il compromesso fra facilità d'uso, automazione, flessibilità e complessità operativa: la scelta va motivata rispetto al tipo di progetto.

Definizioni, passaggi ed esempi presenti nella slide

Metodo Pro Contro

Manuale (FTP, SSH) Facile da iniziare Rischi alti, poco scalabile

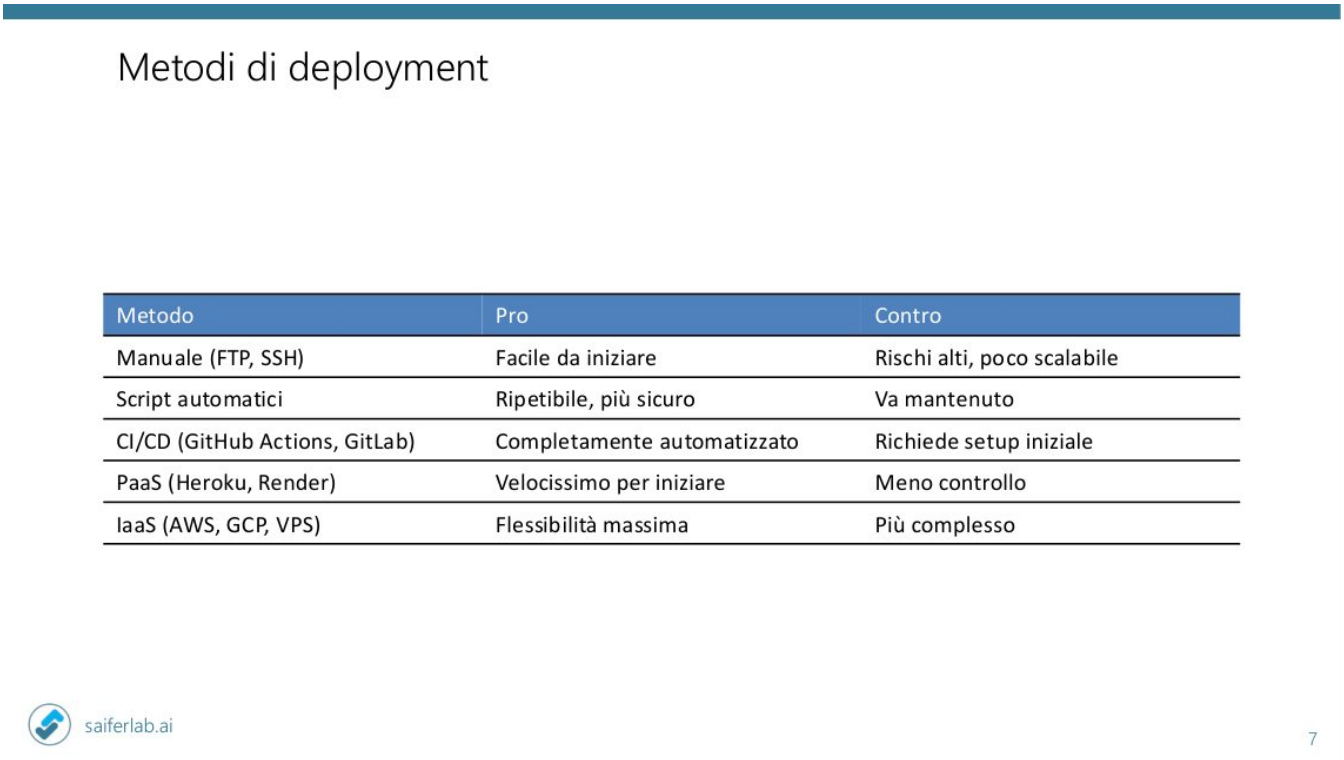
Script automatici Ripetibile, più sicuro Va mantenuto

CI/CD (GitHub Actions, GitLab) Completamente automatizzato Richiede setup iniziale

PaaS (Heroku, Render) Velocissimo per iniziare Meno controllo

IaaS (AWS, GCP, VPS) Flessibilità massima Più complesso

SLIDE ORIGINALE · diagrammi, tabelle e codice



Metodi di deployment

Metodo	Pro	Contro
Manuale (FTP, SSH)	Facile da iniziare	Rischi alti, poco scalabile
Script automatici	Ripetibile, più sicuro	Va mantenuto
CI/CD (GitHub Actions, GitLab)	Completamente automatizzato	Richiede setup iniziale
PaaS (Heroku, Render)	Velocissimo per iniziare	Meno controllo
IaaS (AWS, GCP, VPS)	Flessibilità massima	Più complesso

saiferlab.ai

7

11 / SLIDE 008 Virtualizzazione e container

Lezione: Deployment, container e CI/CD

Fonte: 11_deployment.pdf, pagina 8 di 13

Spiegazione per studiare

Un container racchiude l'applicazione e ciò che serve alla sua esecuzione in un ambiente isolato. Il confronto con le macchine virtuali evidenzia leggerezza e portabilità. Il Dockerfile dell'esempio indica immagine di partenza, cartella di lavoro, copia dei file, installazione dipendenze e comando di avvio.

Definizioni, passaggi ed esempi presenti nella slide

• Container = ambiente isolato con

tutto ciò che serve per eseguire l’app

• Differenze da VM:

• Leggero

• Veloce da avviare

• Più portabile

Esempio Dockerfile (file di configurazione):

```
FROM node:20-alpine
WORKDIR /app
COPY . .
RUN npm install
CMD ["npm", "start"]
```

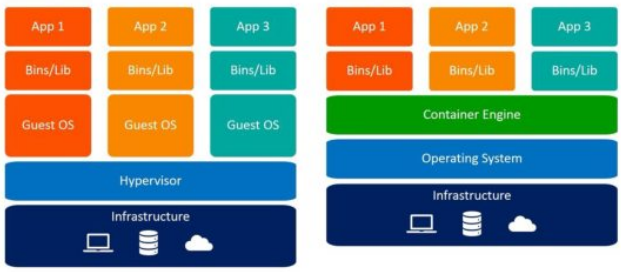
SLIDE ORIGINALE · diagrammi, tabelle e codice

Virtualizzazione e container

- Container = ambiente isolato con tutto ciò che serve per eseguire l’app
- Differenze da VM:
  - Leggero
  - Veloce da avviare
  - Più portabile

Esempio Dockerfile (file di configurazione):

```
FROM node:20-alpine
WORKDIR /app
COPY . .
RUN npm install
CMD ["npm", "start"]
```



Virtual Machines Containers

saiferlab.ai

8

11 / SLIDE 009 Ambienti multi-container: docker-compose

Lezione: Deployment, container e CI/CD
Fonte: 11_deployment.pdf, pagina 9 di 13

Spiegazione per studiare

Docker Compose descrive in YAML più servizi da avviare insieme. L'esempio configura un container Web e uno per PostgreSQL, con porte e variabili d'ambiente. Il file serve quindi a specificare come i componenti cooperano nell'ambiente di esecuzione.

Definizioni, passaggi ed esempi presenti nella slide

Docker-compose è uno strumento per avviare applicazioni multi-container Facilita la gestione dei servizi, la scalabilità, e le connessioni grazie a file di configurazione YAML

```
version: '3.8'
services:
```

web:

build: . ports:

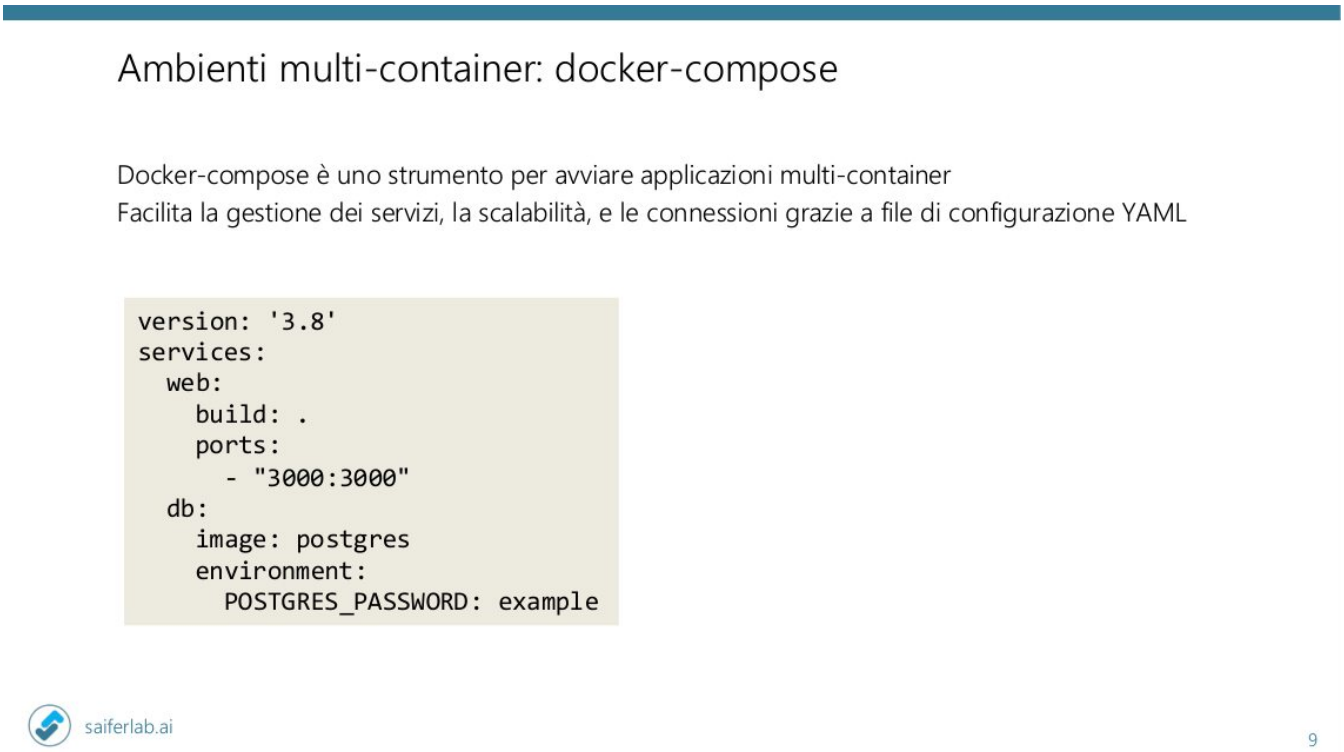
- "3000:3000"

db:

image: postgres environment:

POSTGRES_PASSWORD: example

SLIDE ORIGINALE · diagrammi, tabelle e codice



11 / SLIDE 010 Continuous Integration / Cont. Deployment e Delivery (CI/CD)

Lezione: Deployment, container e CI/CD
Fonte: 11_deployment.pdf, pagina 10 di 13

Spiegazione per studiare

La Continuous Integration automatizza build e test; la Continuous Delivery prepara una versione per il rilascio; la Continuous Deployment arriva a pubblicarla automaticamente. L'esempio GitHub Actions mostra un workflow attivato da `push` che esegue alcuni comandi Docker.

Definizioni, passaggi ed esempi presenti nella slide

• Continuous Integration (CI): build, test, e integrazione di modifiche al codice in un repository condiviso

• Continuous Delivery (CD): consegna automaticamente le modifiche del codice all’ambiente di produzione per essere approvate e messe in funzione

• Continuous Deployment (CD): invia il codice automaticamente ai clienti

Messi insieme, formano una pipeline di CI/CD, ovvero un flusso di lavoro che permette agli sviluppatori di risparmiare tempo, automatizzando i task ripetitivi

```
on: [push]
jobs:
```

deploy:

```
runs-on: ubuntu-latest
steps:
```

• uses: actions/checkout@v2

• run: docker build -t myapp .

• run: docker run myapp

SLIDE ORIGINALE · diagrammi, tabelle e codice

Continuous Integration / Cont. Deployment e Delivery (CI/CD)

- Continuous Integration (CI): build, test, e integrazione di modifiche al codice in un repository condiviso
- Continuous Delivery (CD): consegna automaticamente le modifiche del codice all'ambiente di produzione per essere approvate e messe in funzione
- Continuous Deployment (CD): invia il codice automaticamente ai clienti

Messi insieme, formano una pipeline di CI/CD, ovvero un flusso di lavoro che permette agli sviluppatori di risparmiare tempo, automatizzando i task ripetitivi

```
on: [push]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - run: docker build -t myapp .
      - run: docker run myapp
```



saiferlab.ai

10

11 / SLIDE 011 Configurazione e .env

Lezione: Deployment, container e CI/CD

Fonte: 11_deployment.pdf, pagina 11 di 13

Spiegazione per studiare

Le credenziali e altri valori sensibili non devono essere inseriti direttamente nei sorgenti né pubblicati nel sistema di versionamento. Un file `.env` è presentato come metodo per impostare variabili di ambiente; la slide cita librerie che ne facilitano la lettura.

Definizioni, passaggi ed esempi presenti nella slide

Tra le best practice di sviluppo e configurazione per deployment, uno degli aspetti più importanti è quello di non scrivere mai credenziali nel codice

Si usano file di configurazione, di solito per convenzione chiamati .env (dot-env), per variabili sensibili

• impostano variabili di ambiente, ovvero relative solo all'ambiente in cui saranno usate (non

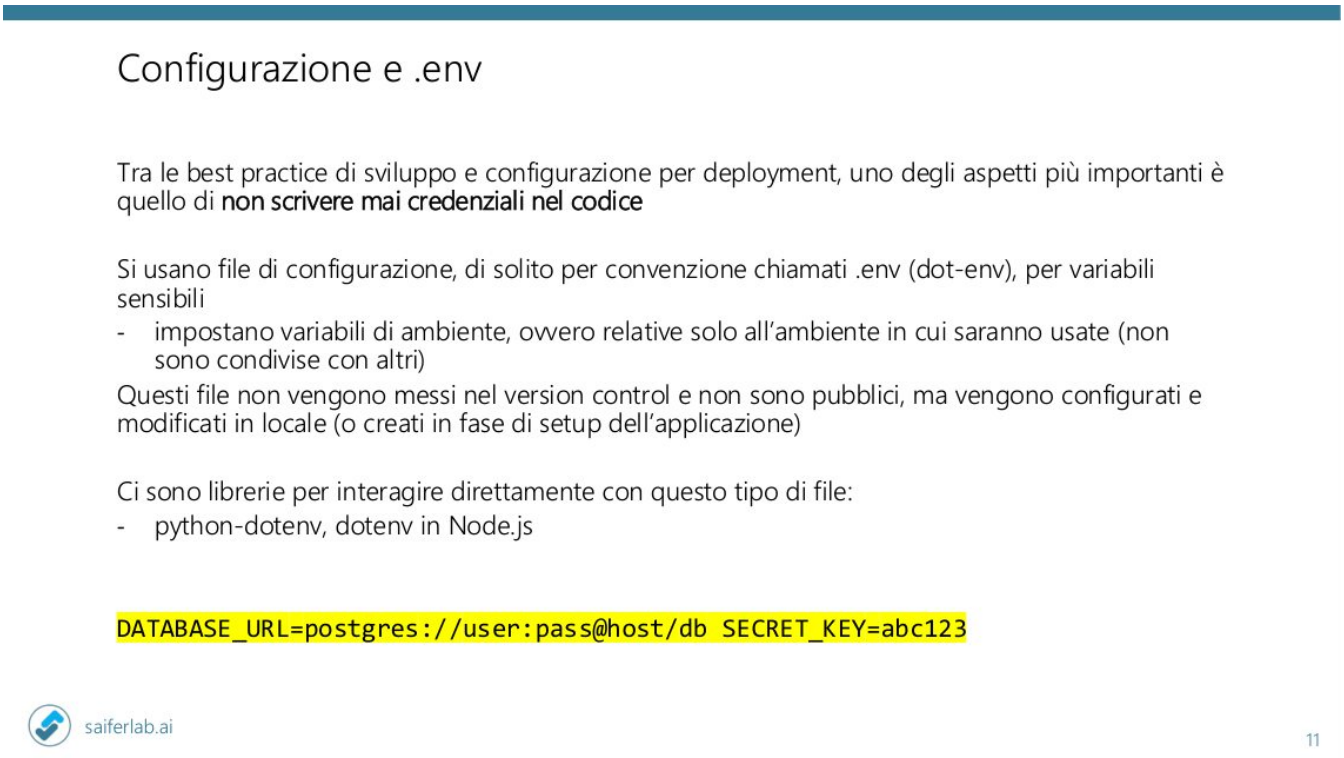
sono condivise con altri) Questi file non vengono messi nel version control e non sono pubblici, ma vengono configurati e modificati in locale (o creati in fase di setup dell'applicazione)

Ci sono librerie per interagire direttamente con questo tipo di file:

• python-dotenv, dotenv in Node.js

```
DATABASE_URL=postgres://user:pass@host/db SECRET_KEY=abc123
```

SLIDE ORIGINALE · diagrammi, tabelle e codice



11 / SLIDE 012 Sicurezza in produzione

Lezione: Deployment, container e CI/CD

Fonte: 11_deployment.pdf, pagina 12 di 13

Spiegazione per studiare

La sicurezza in produzione richiede connessioni HTTPS, limiti sul numero di richieste, validazione degli input, permessi minimi e backup periodici. Queste misure proteggono un servizio già distribuito, dove gli errori possono interessare utenti e dati reali.

Definizioni, passaggi ed esempi presenti nella slide

- HTTPS sempre (Let's Encrypt)
- Rate limiting e protezione da DoS
- Validazione input lato server
- Accessi e permessi ben definiti
- Backup regolari del DB

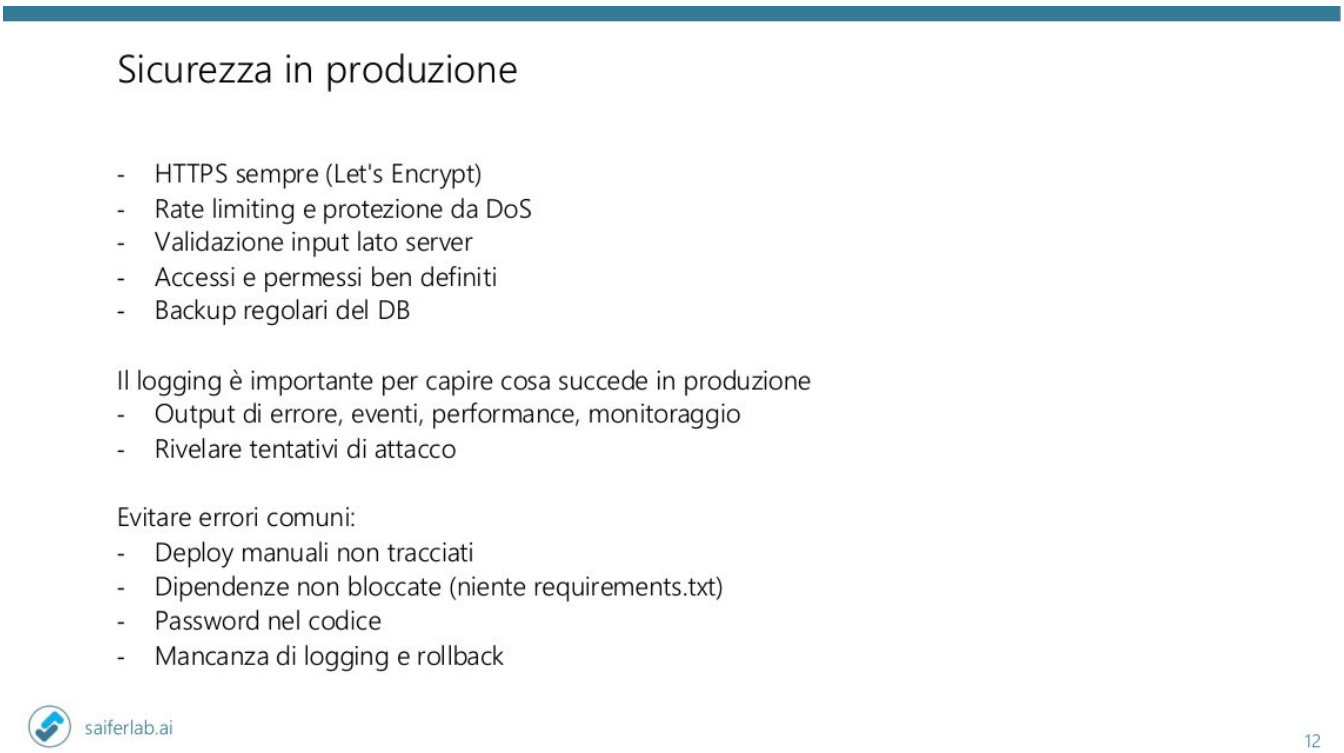
Il logging è importante per capire cosa succede in produzione

- Output di errore, eventi, performance, monitoraggio
- Rivelare tentativi di attacco

Evitare errori comuni:

- Deploy manuali non tracciati
- Dipendenze non bloccate (niente requirements.txt)
- Password nel codice
- Mancanza di logging e rollback

SLIDE ORIGINALE · diagrammi, tabelle e codice



11 / SLIDE 013 Esercitazione con docker-compose

Lezione: Deployment, container e CI/CD

Fonte: 11_deployment.pdf, pagina 13 di 13

Spiegazione per studiare

L'esercitazione finale propone Docker Compose per un sistema che usa un proxy server: un intermediario fra client e servizio a valle. Il materiale rimanda a un repository per il codice pratico; la slide introduce l'attività senza svilupparne tutte le istruzioni.

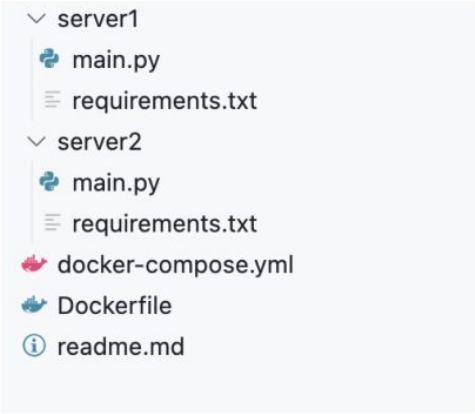
Definizioni, passaggi ed esempi presenti nella slide

Implementeremo un proxy server (un server intermediario che fa da interfaccia tra l'utente e un altro server)

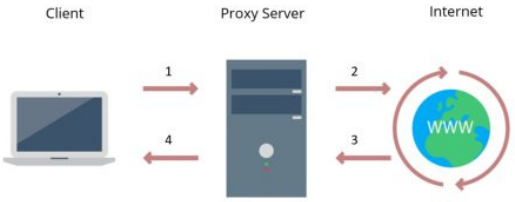
Repository GitHub relativo a docker-compose-tutorial

SLIDE ORIGINALE · diagrammi, tabelle e codice

Esercitazione con docker-compose



Implementeremo un proxy server (un server intermediario che fa da interfaccia tra l'utente e un altro server)



saiferlab.ai

13